

Progress Report

Benchmarking Open-Source Distributed Property-Graph Databases with Portable Mixed Workloads under Elastic Scale-Out

Mustafa Can Çalışkan, 504251544
BLG 564E — Advanced Database Systems
Istanbul Technical University

May 2026

Overview

This document provides an overview of the method, data sources, technologies and experimental approach for my comparative evaluation of five open - source distributed property graph database systems. My research will assess these systems based on a flexible protocol that includes both transactional and analytical graph operations and a simulated node addition to the cluster while mixed operations are occurring. This isn't a proposal of a new database system, the principal contribution of this study is the portable evaluation protocol itself, which enables a comparison between multiple systems that are not standardized according to any single official specification but instead use a universal protocol. It can be easily used with systems having disparate query languages, data storage paradigms, cluster configurations and other differences, through the exploitation of standard components from public LDBC workloads. The main question I'm trying to answer is how these systems cope with the throughput and latency demands imposed by mixed workloads while minimizing cross - workload interference and how this balancing shifts as another node joins the cluster while the operations continue.

1 Systems Under Test

Five systems are selected because of their architectural diversity and their open source availability. They are organized into two benchmarking tiers, so that the experimental depth can be more focused on three systems while keeping the cross-system comparison not below five candidates.

System	Query language	Client library	Tier
NebulaGraph	nGQL	nebula3-python	T1 (full coverage)
ArangoDB 3.11	AQL	python-arango	T1 (full coverage)
Dgraph	DQL	pydgraph	T1 (full coverage)
JanusGraph	Gremlin	gremlinpython	T2 (baseline only)
Apache HugeGraph	Gremlin / REST	gremlinpython, httpx	T2 (baseline only)

Tier-1 systems, as illustrated in Table 1, carry out all the use cases defined in Section 4. Tier-2 systems only perform the transactional - only baseline scenario (S1) due to their demanding deployment complexity. JanusGraph requires a Cassandra storage backend, with an option for Elasticsearch indexing, whereas HugeGraph needs a backend like Cassandra or RocksDB. It was impractical within the project's scope to test all scenarios for Tier-2. Both Tier-2 systems use a shared Cassandra instance spread across three machines, operating in non - concurrent executions to minimize interference. ArangoDB is locked to version 3.11, a version preceding the software licensing alteration.

Image tag and client library version choices are fully specified within the benchmark's repository. Actual image digests will be pinned at the official project launch.

2 Datasets

Three public LDBC-family benchmark datasets are used in the study. Each one is generated by its official data generator.

Dataset	Domain	SF	Role in study
LDBC SNB Interactive v2	Social network	1, (3)	Primary dataset; transactional, analytical, mixed and scale-out scenarios.
LDBC SNB Business Intelligence	Social network	1	Analytical-only baseline source.
LDBC FinBench	Financial transactions	1	Domain-generalization dataset for transactional workload outside the social-network domain.

The Interactive v2 and BI data are generated by the LDBC SNB Datagen (Spark edition), which is provided by LDBC as a Docker image. The FinBench data is created by the LDBC FinBench Datagen, but LDBC doesn't offer a pre - built Docker image, so only the source code is public. The FinBench benchmark repository has a multi - stage Dockerfile, which builds the FinBench Datagen jar from upstream source code and bundles it with a Spark runtime image. As long as the cluster has internet access, you can produce the data reproducibly. The LDBC SNB data consists of Person, Forum, Post, Comment and Tag entities interconnected via relationships like friendship, membership, authorship, replies and tagging. The LDBC FinBench dataset depicts a financial transaction graph, consisting of Person, Company, Account and Loan entities connected by relationships of transfer, deposit, withdrawal and loan application. The data is created on the benchmark client node and then loaded onto each SUT system via its client's specific native client bulk - insert API in batches (INSERT statements for nGQL, `import.bulk` for AQL, JSON mutations for DQL and batched `addV/addE` for Gremlin). The bulk - load time is excluded from the performance measurements.

3 Workload Subset and Query Translation

The Portable Sample includes a workload subset representative of each benchmark workload that allows that workload to be run across different query languages and systems with different feature support. For the transactional subset that S1 and S3 use, I defined a sample composed of queries that can be expressed in portable nGQL, AQL, DQL and Gremlin. It's a subset of LDBC SNB Interactive v2 short reads *IS1–IS7*, selected complex reads *IC1, IC3, IC5, IC6* and write queries *IU1, IU2, IU6* in the LDBC SNB Interactive v2 update stream. I restricted the analytical workload (LDBC SNB BI power - class read query workload *Q1, Q2, Q6, Q11*) and the financial transactional workload (LDBC FinBench simple - read query workload *SR1–SR6* and write workload *W1–W3*) to nGQL, AQL and DQL because Tier-2 systems don't support these scenarios. For each system, I manually translate its corresponding workload query or workload - related workload and I check that its structure and input types conform to the LDBC specification. I don't fully check semantic equivalence to a reference LDBC implementation because this is planned as future work (described in Section 10). The query translation strings are stored as is in the benchmark repository to enable human inspection.

4 Scenarios

Five scenarios are defined in this study. Scenario S1 serves as the cross-tier baseline and it is executed by all five systems. Scenarios S2–S5 are executed only by Tier-1 systems.

ID	Name	Workload composition	Dataset(s)	Systems
S1	Transactional-only	IS1–IS7, IU1, IU2, IU6	Interactive v2	all five
S2	Analytical-only	IC1, IC3, IC5, IC6, BI Q1, Q2, Q6, Q11	Interactive v2, BI	Tier 1
S3	Mixed	70% IS, 20% IC, 10% IU (request ratio)	Interactive v2	Tier 1
S4	Financial tx.	FinBench SR1–SR6, W1–W3	FinBench	Tier 1
S5	Mixed + scale-out	Same as S3 + one node joins at $t=300$ s	Interactive v2	Tier 1

5 Workload Protocol

I execute five scenarios and each scenario run is structured with the following sequence of steps. (i) I use a Fabric task that spins up a cluster, first by running `docker compose` on the allocated nodes and then running a system-dependent readiness probe. (ii) Then I use the native client loader of the target system to load the scenario’s dataset in bulk. (iii) Next I run post - load validators that check against five known - answer queries and ensure that the dataset loaded properly. (iv) Then I run a five - minute warm - up phase, where the workload runs but I discard all of the metrics samples. (v) After the warm - up, I run the measurement phase for fifteen minutes (or twenty minutes for scenario S5). (vi) Only in scenario S5, I trigger a scale - out event at time $t=300$ s into the measurement phase. (vii) Finally, I teardown the cluster and rsync result artifacts to the benchmark client node.

I set the client concurrency to sixteen asynchronous workers per scenario, except for scenario S2, which has eight. I do this because the analytical queries for S2 hold per - request resources for longer than those of other scenarios, and increasing the worker count for S2 could saturate the cluster before I observe the distribution effects I’m looking for. The workload is closed - loop: clients issue the next query only once they have the response of the current query, and I don’t add any inter - request think time. The set of queries to choose from is defined by the scenario’s mix and a weighted random sampler. The parameters for each query are chosen from a per - query pool that reflects the vertex distribution of the dataset than the curated parameter selection in the LDBC SNB Driver (as documented in Section 10). This pool is created through a seeded pseudo - random sampler based on the vertex CSVs in the dataset. The seed is a function of the (system, scenario, repetition index) tuple, this makes parameters deterministic per repetition but different across repetitions. I set a per - query timeout to sixty seconds. Queries exceeding this timeout are marked as failed and excluded from latency statistics, while being counted towards the failed query rate. For each (system, scenario) configuration, I run three repetitions. The reported statistics are the mean across the repetitions, with standard deviation as the error bar.

6 Metrics

- **Throughput** (ops/s): overall and per query-class, sampled at 1 s intervals.
- **Latency** (μ s): per-request, aggregated in an HDR histogram; percentiles $p50$, $p95$, $p99$, $p99.9$ are reported per query class.
- **Mixed-workload interference**: $I = 1 - T_{\text{mixed}} / \sum_c w_c T_c^{\text{iso}}$, where T_c^{iso} is the throughput of class c measured in isolation (S1 or S2) and w_c is its weight in the mix.
- **Scale-out stabilization time**: elapsed time between the scale-out trigger and the first instant where the rolling-average throughput re-enters the band $[0.95, 1.05] \cdot T_{\text{pre-event}}$ and stays inside that band for at least sixty seconds.
- **Scale-out throughput dip**: the maximum relative throughput reduction observed during the rebalance window.
- **Failed-query rate**: fraction of queries that return an error or exceed the sixty-second timeout.
- **Per-node resource usage**: CPU, memory, disk I/O and network I/O, sampled at 5 s intervals through direct `/proc` reads (`stat`, `meminfo`, `diskstats`, `net/dev`) which are streamed over SSH from each SUT node. A local `psutil` sample is also added for the client node.

7 Experimental Environment

The experiments are run on a five - node compute cluster that HAVELSAN provides. I have administrator privileges and a Linux - based operating system on each node. The nodes form a cluster that can address each other via a private network using their hostnames. Each node features an x86.64 CPU that has 8–16 physical cores per socket, at least 32 GiB of RAM, a local SSD for storage and a 1–10 Gbit/s network interface. The exact specifics for each node will be reported in the experimental - setup chapter of the paper. For example, it is planned to mention the type of CPUs, the kernel version in each Linux distribution and if I found that the nodes aren’t identical class, but were assumed equal class. The nodes are partitioned with the following roles:

Node	Role
node1	Benchmark driver, metrics collector, orchestrator
node2–node4	Initial three-node SUT cluster
node5	Reserve node, activated during scale-out event (S5)

The SUTs are run as containers on the nodes they’re constrained to, using the `docker compose` files that are installed system - specific. Inter - node actions like copying of container images, startups, shutdowns, passing configurations around and starting scale - out runs, are initiated by Fabric tasks executed on node1 over SSH. I don’t rely on a shared file system. The datasets stay on the client node, and the loaders push the data into each SUT through its native client API. After the test concludes, I collect the result artifacts back to the client node with `rsync`.

Inside this cluster configuration, I set up each system with the described service - role breakdown as laid out in Table 1. The Tier-2 systems use the same Cassandra storage cluster distributed on nodes 2 through 4, JanusGraph and HugeGraph access it on separate keyspaces. Since Tier-2 executions are only planned to run in time - separated intervals, this shared Cassandra resource only needs to support one Tier-2 system at a time.

System	node2–node4 (each)	node5 (S5)
NebulaGraph	metad, stored, graphd	one additional stored
ArangoDB 3.11	agent, coordinator, dbserver	one additional dbserver
Dgraph	one zero, one alpha	one additional alpha
JanusGraph	cassandra, janusgraph-server	—
Apache HugeGraph	cassandra, hugegraph-server	—

Table 1: Service-role distribution per system across cluster nodes.

In scenario S5, the scale - out happens with a system - appropriate call. ArangoDB register its new dbserver via cluster agency join. NebulaGraph add the new stored instance at node5 by using `ADD HOSTS` against the metad service. Dgraph starts the new alpha process with the known zero address, it then automatically adds itself into the existing predicate group. I record the timestamp when the registration call is issued as the scale - out event timestamp and use it to measure scale - out stabilization time and throughput dip (as introduced in Section 6).

8 Software Stack

The benchmark runner, the stats and the output text have all been put together in Python (using 3.11 for it). Apache Parquet is what I used to generate the per - request log files and I relied on pyarrow to accomplish it. I store per - request latency data in HDR histograms using the Python `hdrpy` library, these data are summarized for output purposes. The YAML format is used for the configuration files, with Pydantic v2 used for validation. An `asyncio` application pool relies on both `asyncio` and `anyio`. Tasks in the Fabric cluster management tool orchestrate multi - host operations, including: up the cluster, triggers the cluster scale - out, gathers up the results and finally teardowns the cluster. A table in Section 1 contains all per - system client libraries. Statistics collection utilizes NumPy, SciPy and Pandas, figures are produced with Matplotlib with a consistent paper style: serif font, fixed size and shape, no frivolous details added. A Python script under `analysis/reports/` reads the run artifacts and produces the final report as a Markdown document with figures and CSV summaries. Dataset generation relies on the LDBC SNB Datagen Docker image provided by the organization, alongside an LDBC FinBench Datagen jar that was compiled from an upstream source file using a multi - stage Dockerfile (the FinBench project doesn’t release any images of their own, so this was the only reasonable path forward). The source code of the benchmark is publicly available at <https://github.com/canetizen/pdgraph-bench>, together with all configurations, container specifications and the per - system query translations.

9 Planned Experimental Campaign

My benchmarks comprise 51 test runs in total. There will be 3 tier 1 systems across 5 different test scenarios repeated three times each (45 test runs total). Additionally, I'll run 2 tier 2 systems against scenario S1 3 times each (6 runs total). Each individual run - a full cycle including system setup, data ingestion, validation, warm - up, measurement and cleanup - should run 25-35 minutes long. Every test run saves its artifacts to a separate directory `results/<run_id>/`: per - request log file in Parquet format, per - class HDR latency histograms, per - second throughput buckets in Parquet format, per - node metric samples in Parquet format and an events file in jsonl format capturing phase transitions and any cluster scaling events. Each run can be reproduced by re - running the same scenario configuration on the same repository checkout, because the harness is deterministic for the (system, scenario, repetition index) seed.

10 Threats to Validity (Anticipated)

Several limitations should be taken into account about the research presented. Firstly, the manual translation of Cypher queries to nGQL, AQL, DQL and Gremlin has been checked against the LDBC specification but not a concrete implementation. These are provided to allow readers to judge query accuracy themselves. Secondly, per - query parameters were sampled from the loaded CSV vertex data than using the LDBC SNB Driver's specific parameter file for each query. This increases within - query latency variance but leaves cross - system latency comparison unaffected as all systems consume the same dataset. Thirdly, SF1 is relatively small compared to the scale of systems in production. Thus, I report behaviors observed in this setting than claiming the results extend to larger data sizes. Fourthly, the cluster has only 5 nodes in total, so I could only execute one scaling - out experiment (from 3 to 4 data - serving nodes). Fifthly, since the Tier-2 systems only run scenario S1 because of their deployment complexity (as I explained in Section 1), I narrow down the claims of cross - tier comparison. Finally, bulk loading employs batching over each system's native client API instead of the specific bulk loaders, e. g. , `nebula - importer`, `arangoimport`, `dgraph live`. I do not measure load times, so their absolute performance can't be compared.